Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ
DE L'EMPLOI

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▸ JOUHANS
Nom d'usage ▸ JOUHANS
Prénom ▸ Tommy
Adresse ▸ 1, Avenue de la Visitation
Presbytère
21800 Chevigny-Saint-Sauveur

Titre professionnel visé

Titre RNCP de niveau 5 : Développeur Web et Web Mobile

MODALITE D'ACCES :

☒ Parcours de formation
☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen**.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

http://travail-emploi.gouv.fr/titres-professionnels

Sommaire

Exemples de pratique professionnelle p. 4

Intitulé de l'activité-type n° 1 : Développer la partie front-end d'une application web ou web mobile sécurisée p. 5

- Intitulé de l'exemple n° 1 : Maquettage et réalisation des interfaces utilisateur sécurisées ..... p. 5
- Intitulé de l'exemple n° 2 : Développement de la partie dynamique des interfaces utilisateur ..... p. 6
- Intitulé de l'exemple n° 3 : Installation et configuration de l'environnement de travail ... p. 7

Intitulé de l'activité-type n° 2 : Développer la partie back-end d'une application web ou web mobile sécurisée p. 8

- Intitulé de l'exemple n° 1 : Conception et mise en place de la base de données relationnelle ..... p. 8
- Intitulé de l'exemple n° 2 : Développement des composants métier, accès aux données et sécurisation de l'API ..... p. 9
- Intitulé de l'exemple n° 3 : Documentation du déploiement de l'application ..... p. 10

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 11

Déclaration sur l'honneur p. 12

Documents illustrant la pratique professionnelle *(facultatif)* p. 13

Annexes *(Si le RC le prévoit)* p. 14

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n° 1 ▸ Maquettage et réalisation des interfaces utilisateur sécurisées

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

À partir du recueil des besoins utilisateurs et du persona Marie, gérante de PME non technophile, j'ai réalisé le maquettage des onze écrans de l'application CyberAudit & Solutions sur Figma, en respectant une charte graphique cohérente et en tenant compte de l'accessibilité pour les personnes en situation de handicap. J'ai ensuite développé les interfaces utilisateur statiques correspondantes en HTML sémantique généré par React et Tailwind CSS, selon une approche « mobile first », en respectant les recommandations de sécurité applicables aux interfaces web.

2. Précisez les moyens utilisés :

Figma pour le maquettage des interfaces et le schéma d'enchaînement. React 19, Vite et Tailwind CSS 4 pour le développement des interfaces statiques. Visual Studio Code, Git et GitHub. Le Référentiel Général d'Amélioration de l'Accessibilité (RGAA) et le guide de l'ANSSI pour la sécurisation des interfaces.

3. Avec qui avez-vous travaillé ?

J'ai travaillé en binôme avec James ROUSSEL, qui s'occupe la partie back-end du projet, ainsi qu'avec l'équipe pédagogique de Holberton School Dijon, qui a validé les choix de conception lors des points hebdomadaires.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *CyberAudit & Solutions, projet réalisé en formation à Holberton School Dijon*

Chantier, atelier, service ▸ Maquettage Figma et développement des interfaces statiques de la plateforme CyberAudit & Solutions (pages d'accueil, inscription, connexion, tableaux de bord, formulaire de demande d'audit)

Période d'exercice ▸ Du : *04/05/2026* au : *22/05/2026*

5. Informations complémentaires *(facultatif)*

Le détail technique complet de cette réalisation (schéma d'enchaînement, captures d'écran des maquettes, extraits de code des interfaces statiques) est présenté dans le Dossier de Projet joint.

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n° 2 ▸ **Développement de la partie dynamique des interfaces utilisateur**

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai développé la partie dynamique de CyberAudit & Solutions à partir des interfaces statiques et du dossier de conception de l'API back-end. La gestion de l'authentification repose sur un contexte React global (AuthContext) et un composant de garde d'accès selon le rôle (ProtectedRoute). L'API REST sécurisée est consommée via une instance Axios centralisée avec rafraîchissement automatique des jetons JWT expirés. Le tableau de bord client affiche les demandes d'audit à partir d'une API paginée, avec des badges de statut colorés. J'ai développé le formulaire de soumission en trois étapes, dont le temps de complétion (moins de trois minutes) a été validé par trois tests utilisateurs. Enfin, j'ai mis en place l'assainissement DOMPurify des contenus saisis par l'utilisateur pour prévenir les attaques XSS.

2. Précisez les moyens utilisés :

React 19 (hooks, Context API, React Router 7). Axios pour la consommation de l'API REST et la gestion des jetons JWT. DOMPurify pour l'assainissement anti-XSS. Vitest et React Testing Library pour les tests unitaires des composants et hooks (62 tests). GitHub Actions pour l'intégration continue.

3. Avec qui avez-vous travaillé ?

Coordination étroite avec James ROUSSEL pour synchroniser le contrat d'interface JSON entre le front-end et l'API Django REST Framework, avec revues de code croisées systématiques avant chaque fusion de branche.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *CyberAudit & Solutions, projet réalisé en formation à Holberton School Dijon*

Chantier, atelier, service ▸ Développement de la logique dynamique, de l'authentification et de la consommation de l'API sécurisée de la plateforme CyberAudit & Solutions

Période d'exercice ▸ Du : *25/05/2026* au : *19/06/2026*

5. Informations complémentaires *(facultatif)*

Le détail technique (extraits de code du contexte d'authentification, de l'intercepteur Axios, du composant de garde d'accès) est présenté dans le Dossier de Projet joint, sections 7.5 et 7.6, et en Annexes 5 et 6 du présent dossier

DOSSIER PROFESSIONNEL (DP)

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n° 3 ▸ **Installation et configuration de l'environnement de travail**

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

En début de projet, j'ai installé et configuré mon environnement de travail front-end : Node.js 20, le gestionnaire de paquets npm, l'outil de build Vite, l'éditeur Visual Studio Code avec les extensions ESLint et Prettier, et le client Git. J'ai mis en place la configuration ESLint/Prettier du projet (règles React et react-hooks) ainsi que les scripts du fichier package.json (dev, build, lint, test). J'ai également participé, avec James ROUSSEL, à la configuration de l'environnement Docker utilisé pour reproduire localement la base PostgreSQL et le broker Redis, afin de disposer d'un environnement de développement conforme à la production. Cette installation a fait l'objet d'une documentation technique en français et en anglais pour faciliter l'onboarding.

2. Précisez les moyens utilisés :

Node.js 20, npm, Vite, ESLint, Prettier, Visual Studio Code, Git et GitHub (convention de commits, branches feature/*, revues par pull request). Docker et Docker Compose pour les services PostgreSQL et Redis de test.

3. Avec qui avez-vous travaillé ?

Seul pour la configuration front-end ; avec James ROUSSEL pour la définition commune de l'environnement Docker partagé (services, variables d'environnement, ports).

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *CyberAudit & Solutions, projet réalisé en formation à Holberton School Dijon*

Chantier, atelier, service ▸ Mise en place de l'environnement de travail front-end et de l'environnement Docker partagé

Période d'exercice ▸ Du : *22/06/2026* au : *10/07/2026*

5. Informations complémentaires *(facultatif)*

Le détail (extrait du package.json, règles ESLint, convention de commits) est présenté dans le Dossier de Projet joint, section 7.1.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n° 1 ▸ **Conception et mise en place de la base de données relationnelle**

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

En binôme avec James ROUSSEL, j'ai participé activement à la conception du schéma conceptuel de données de CyberAudit & Solutions lors des sprints de conception (semaines 1 et 2) : identification des sept entités principales et de leurs relations, définition des règles de validation et des contraintes du modèle relationnel PostgreSQL (clés UUID, contraintes d'unicité, cardinalités), et identification conjointe des index nécessaires à la performance des requêtes du tableau de bord (statut, client + date de soumission, unicité demande-rapport). J'ai ensuite vérifié, du point de vue front-end, la conformité du schéma aux besoins d'affichage des tableaux de bord client et administrateur, via les contrats d'interface JSON définis en commun.

2. Précisez les moyens utilisés :

PostgreSQL 15. Django 5.1 et son système de migrations pour la création et l'évolution du schéma physique. Un outil de modélisation pour le schéma entité-association. Git et GitHub pour la revue croisée des migrations avant application.

3. Avec qui avez-vous travaillé ?

Conception menée conjointement avec James ROUSSEL, responsable de l'implémentation technique du modèle sous Django.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *CyberAudit & Solutions, projet réalisé en formation à Holberton School Dijon*

Chantier, atelier, service ▸ Conception du schéma conceptuel et physique de la base de données relationnelle de la plateforme CyberAudit & Solutions

Période d'exercice ▸ Du : *06/05/2026* au : *22/05/2026*

5. Informations complémentaires *(facultatif)*

Le schéma physique complet, le tableau des entités et relations, et un extrait de migration Django sont présentés dans le Dossier de Projet joint, section 8.1, et en Annexes 3 et 7 du présent dossier.

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n° 2 ▸ **Développement des composants métier, accès aux données et sécurisation de l'API**

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai contribué à la définition des exigences de sécurité de l'API REST consommée par le front-end : durée de vie des jetons JWT, rotation, contrôle d'accès par rôle (RBAC) et limitation du débit sur les routes sensibles.

2. Précisez les moyens utilisés :

Django REST Framework pour l'exposition de l'API REST, SimpleJWT pour l'authentification par jeton. Postman pour les tests manuels de l'API et des scénarios de sécurité. Pytest pour les tests unitaires et d'intégration back-end (131 tests). GitHub Actions pour l'intégration continue du back-end.

3. Avec qui avez-vous travaillé ?

Réalisation menée en binôme avec James ROUSSEL, responsable de l'implémentation back-end, avec définition commune des contrats d'API et revues de code croisées systématiques avant tout développement.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *CyberAudit & Solutions, projet réalisé en formation à Holberton School Dijon*

Chantier, atelier, service ▸ Définition fonctionnelle des composants métier et des exigences de sécurité de l'API REST

Période d'exercice ▸ Du : *25/05/2026* au : *19/06/2026*

5. Informations complémentaires *(facultatif)*

Le détail (extraits de code des composants métier, configuration JWT, résultats des tests de sécurité) est présenté dans le Dossier de Projet joint, sections 8.3, 8.4 et 10.3.

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n° 3 ▸ **Documentation du déploiement de l'application**

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai rédigé la procédure de déploiement du front-end sur Vercel (connexion du dépôt GitHub, configuration des variables d'environnement, en-têtes de sécurité HTTP dans vercel.json : HSTS, X-Frame-Options, Content-Security-Policy) et j'ai participé, avec James ROUSSEL, à la définition du pipeline d'intégration continue GitHub Actions commun aux deux applications (lint, tests, build, déploiement automatique) déclenché à chaque poussée sur les branches main et develop.

2. Précisez les moyens utilisés :

GitHub Actions, Vercel et son fichier de configuration vercel.json, variables d'environnement, fichier « .env.example ». Sources de veille : OWASP Top 10, CERT-FR, alertes GitHub Dependabot.

3. Avec qui avez-vous travaillé ?

Réalisation menée en binôme avec James ROUSSEL, responsable de l'implémentation back-end, avec définition commune des contrats d'API et revues de code croisées systématiques avant tout développement.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *CyberAudit & Solutions, projet réalisé en formation à Holberton School Dijon*

Chantier, atelier, service ▸ Mise en place du déploiement continu et de la configuration de production du front-end

Période d'exercice ▸ Du : *22/06/2026* au : *28/06/2026*

5. Informations complémentaires *(facultatif)*

Le détail (extrait du pipeline GitHub Actions, configuration *vercel.json*, exemple de veille tracée) est présenté dans le Dossier de Projet joint, section 8.5.

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Foundations of Computer Science	Holberton School à Dijon	17/07/2026
BTS Informatique et réseaux pour l'industrie et services technique	Lycée Saint Marie Fenelon à Lons-le-Saunier	07/07/2015
Bac Professionnel Systèmes Electronique et Réseaux pour l'Industrie	Lycée Professionnel Saint Joseph à Bourg-en-Bresse	05/07/2012

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] Monsieur Tommy JOUHANS ,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à Dijon le 18/10/2026

pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Dossier de Projet : CyberAudit & Solutions (joint au présent dossier professionnel)
Dépôt de code source : https:/github.com/Tommy-JOUHANS/portfolio
Documentation technique du projet (architecture, base de données, sécurité, API) : https:/github.com/Tommy-JOUHANS/portfolio/tree/main/Stages/Stage-3
Documentation d'exécution et de tests (Stage 4 : Sprints, tests, déploiement) : https:/github.com/Tommy-JOUHANS/portfolio/tree/main/Stages/Stage-4
Vidéo de démonstration de l'application : https://www.youtube.com/watch?v=jxrV9IZSC70

ANNEXES

(Si le RC le prévoit)

Dossier Professionnel : CyberAudit & Solutions

Ce document rassemble les pièces annexes citées dans les trois exemples de pratique professionnelle du Dossier Professionnel : schéma de navigation, captures d'écran des maquettes, schémas techniques et extraits de code. Chaque annexe peut être copiée directement dans la section « Annexes » du Dossier Professionnel officiel.

Annexe 1 : Arborescence des composants et navigation

Schéma illustrant l'enchaînement des composants structurants de l'application (Header, Sidebar, Router) et la logique de navigation entre les pages, référencé dans l'Exemple n°1 (Activité-type 1).

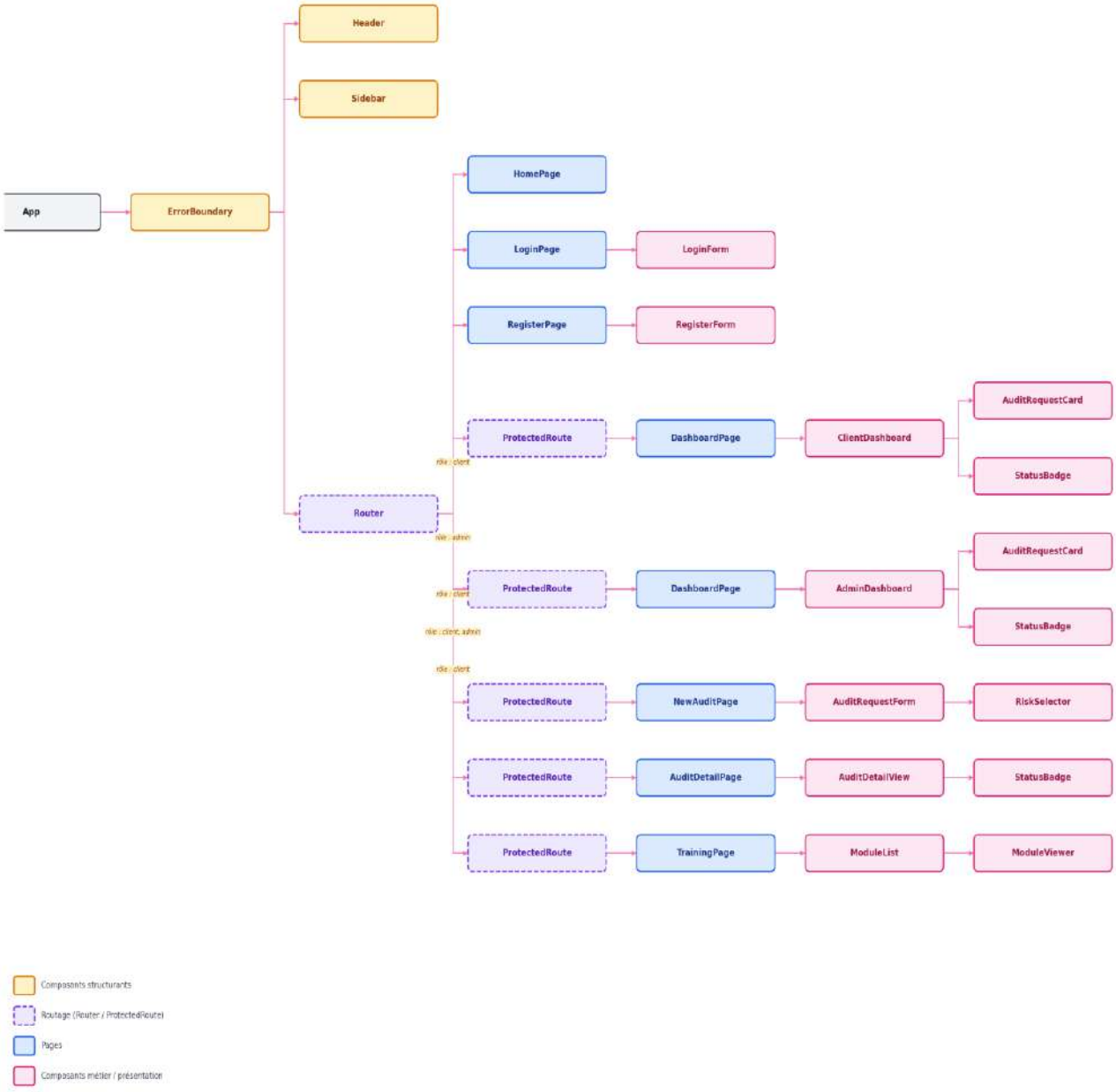
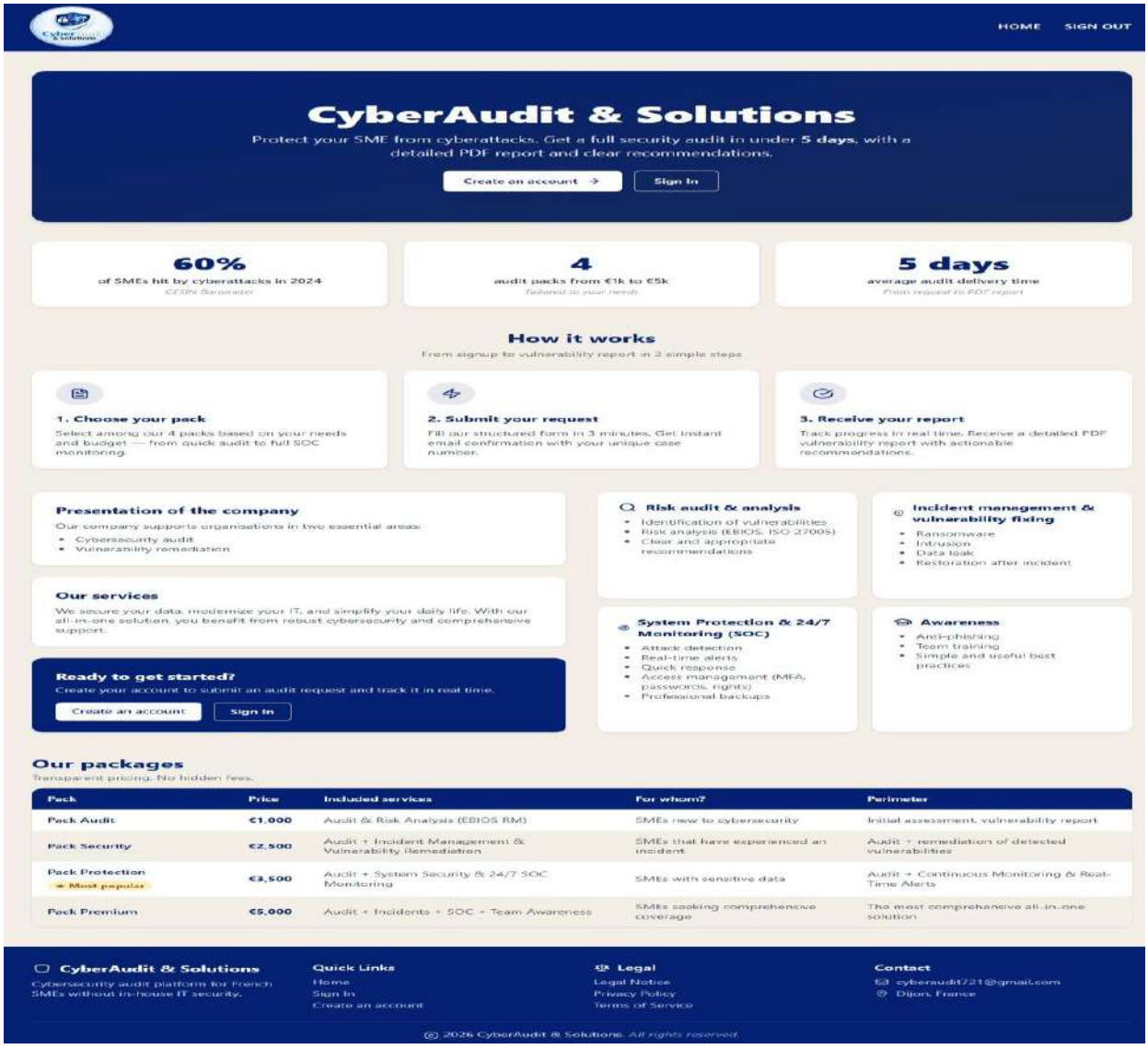


Figure 1 : Arborescence des composants et navigation de CyberAudit & Solutions

Côté front-end, l'application s'organise autour d'un routeur protégé : chaque page sensible passe par ProtectedRoute, qui vérifie le rôle avant d'afficher le tableau de bord client ou administrateur. Tous les composants métier partagent une couche d'accès à l'API commune et des composants réutilisables comme le badge de statut ou la jauge de score de sécurité.

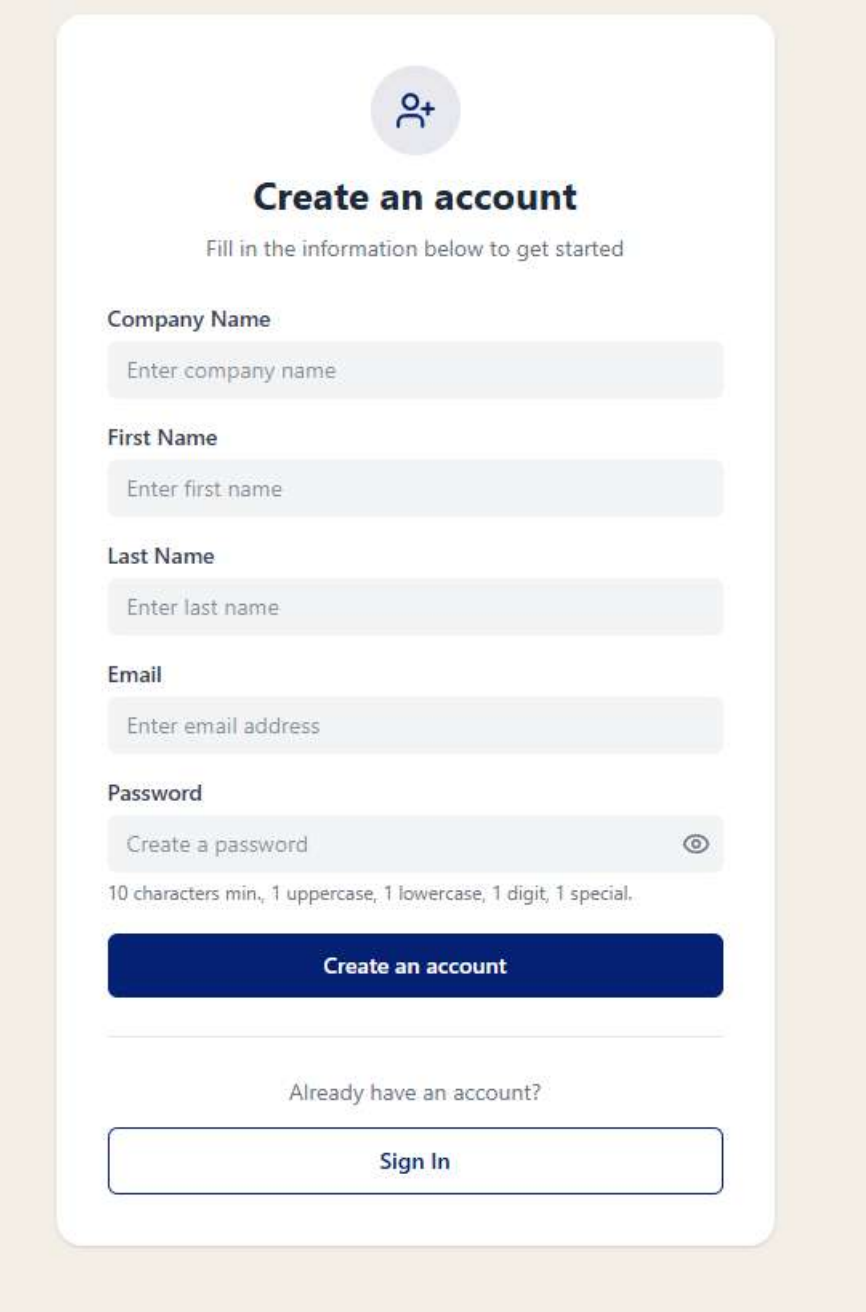
Annexe 2 : Captures d'écran des onze maquettes de l'application

Onze écrans ont été maquettés sur Figma dès le premier sprint, avant tout développement, conformément à la démarche de conception centrée utilisateur. Chaque écran a été conçu en version desktop et en version mobile (Breakpoints Tailwind sm/md/lg, approche « mobile first »), conformément à l'exigence d'adaptation web et web mobile du référentiel ; les captures ci-dessous présentent la version desktop des maquettes principales, la version mobile étant vérifiable sur le fichier Figma du projet et sur la démonstration en ligne. Chaque maquette a été pensée en priorité pour la persona Marie, avec un objectif constant de simplicité et de minimalité des affichages.



Écran 1 : page d'accueil publique « Présentation de l'entreprise, des services et des formules, avec appel à l'action vers l'inscription ».

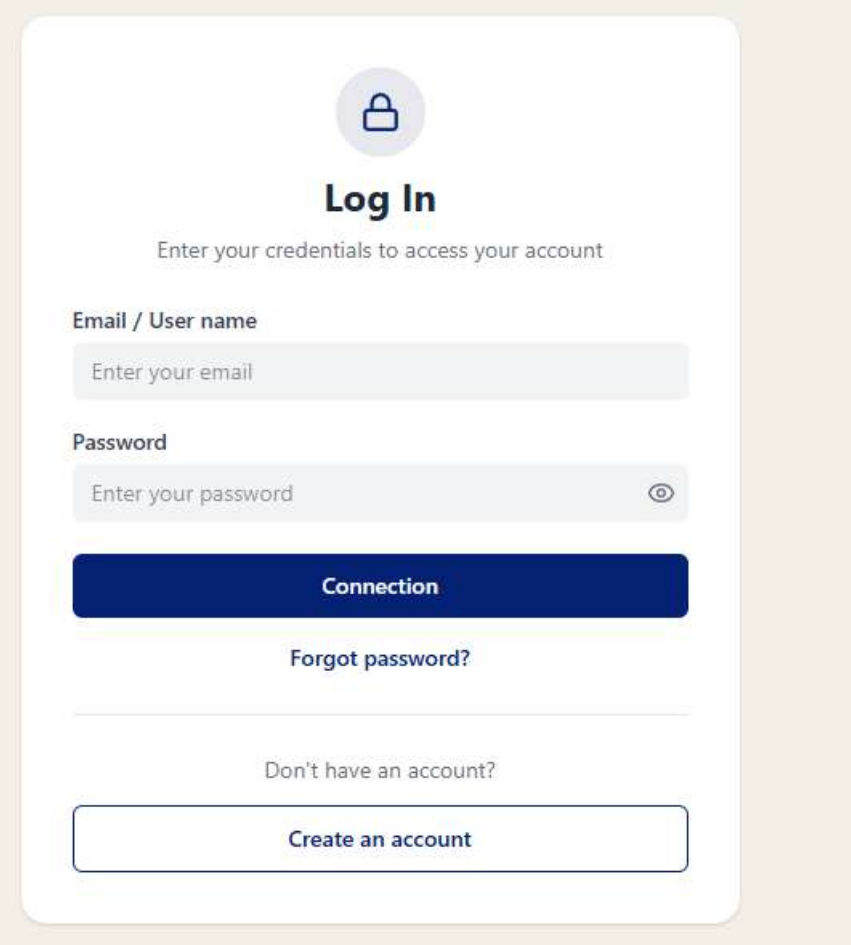
La page d'accueil de la plateforme CyberAudit & Solutions présente l'entreprise, ses services et les packs proposés, taillés pour les entrepreneurs de moins de 50 collaborateurs. Il suffit de se rendre dans l'onglet « Sign in » pour que la cliente (Marie) puisse créer un compte et se connecter.



Écran 2 : Inscription avec un courriel, un mot de passe, un consentement RGPD/CGU. Règles de validation détaillées en section 7.4.

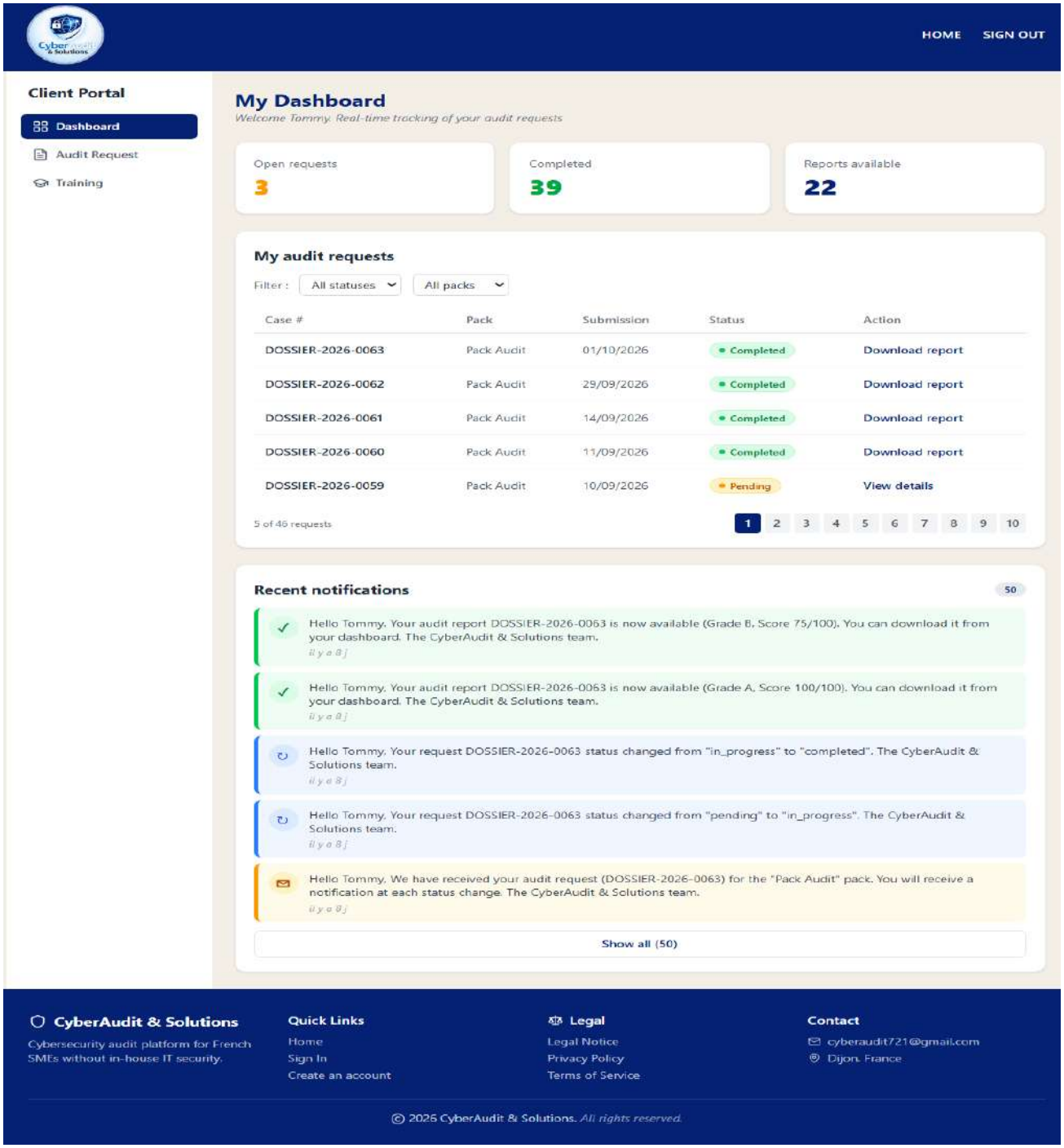
Après avoir cliqué sur « Créer un compte », la cliente saisit le nom de son entreprise, son prénom, son nom, son courriel et son mot de passe (haché côté serveur), et coche une case de consentement RGPD/CGU obligatoire à la soumission du formulaire.

Règles de remplissage du formulaire : chaque champ contient au maximum 50 caractères ; si un champ est manquant, le message d'erreur « Veuillez renseigner les informations du formulaire » s'affiche ; le format courriel (nom@domaine) est requis ; le mot de passe doit comporter une minuscule, une majuscule, un chiffre, un caractère spécial et dix caractères minimum.



Écran 3 : Connexion par courriel et mot de passe, lien vers la création de compte.

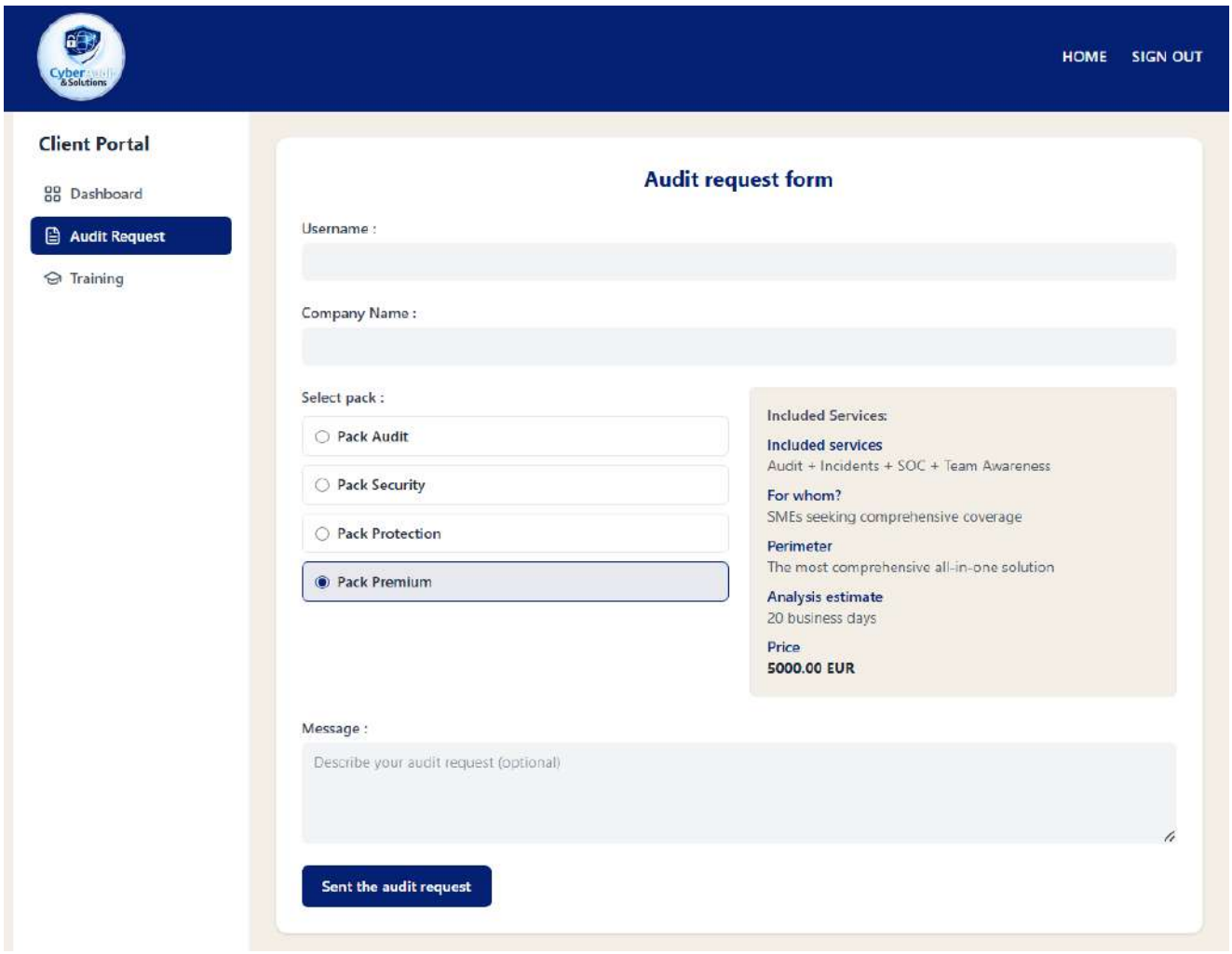
La cliente peut se connecter avec son compte ; sinon, elle clique sur « Créer un compte ». Les liens « Courriel oublié » et « Mot de passe oublié » sont prévus dans la maquette (fonctionnalité de priorité Should have, à développer ultérieurement).



Écran 4 : Tableau de bord client « Liste des demandes avec statut coloré, fil de notifications et action contextuelle par ligne (voir le détail ou télécharger le rapport disponible) ».

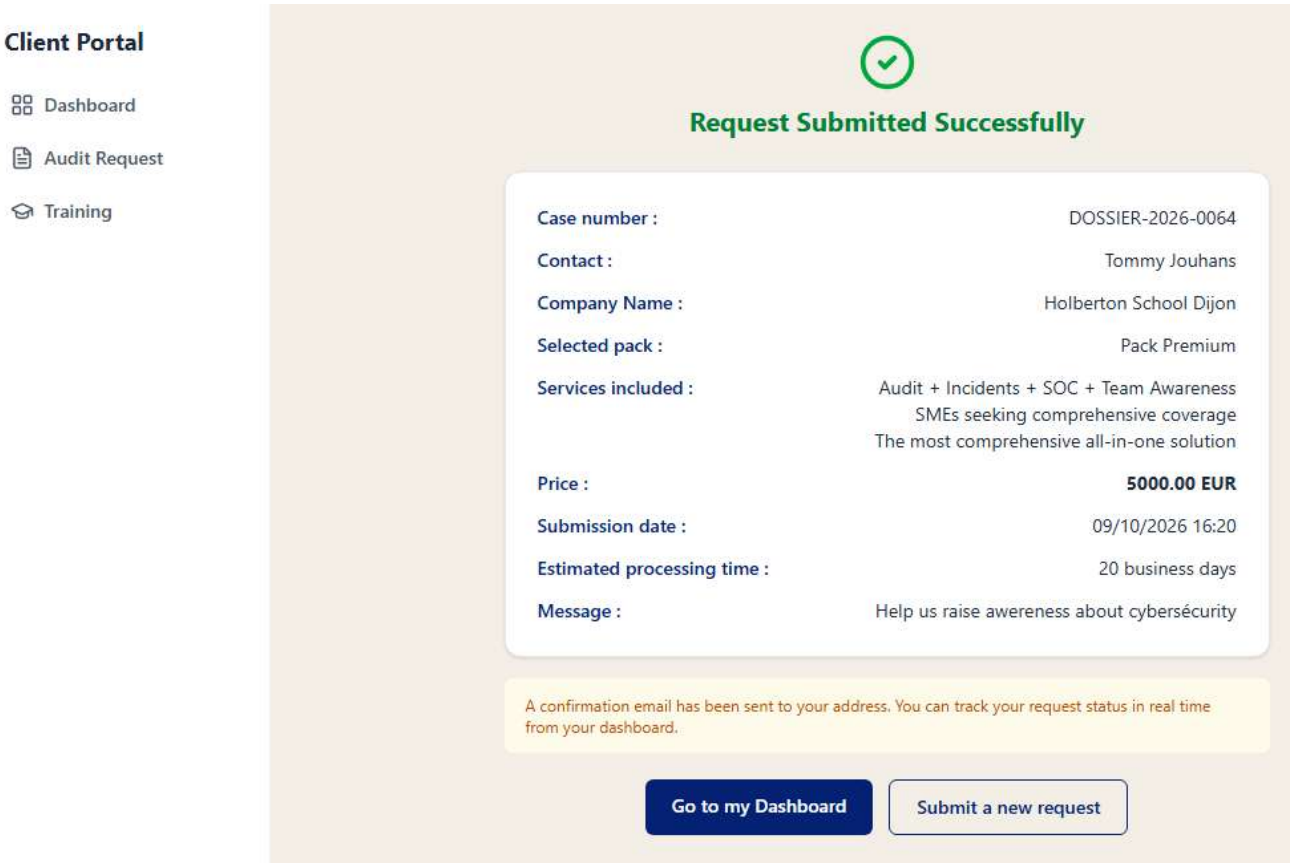
Une fois connectée, la cliente consulte son tableau de bord pour suivre l'avancement de chacune de ses demandes d'audit (En attente / En cours / Terminée / Archivée), affichées sous forme de liste avec un badge de statut coloré et un fil de notifications récentes (par exemple un changement de statut). La colonne « Action » propose « Voir les détails » pour une demande en cours (écran 6) ou « Télécharger le rapport » lorsque le statut est Terminé (écran 11, il s'agit de la même interface, la fonctionnalité affichée

dépendant du statut de la demande). L'onglet « Audit Request » permet de créer une nouvelle demande (écran 5).



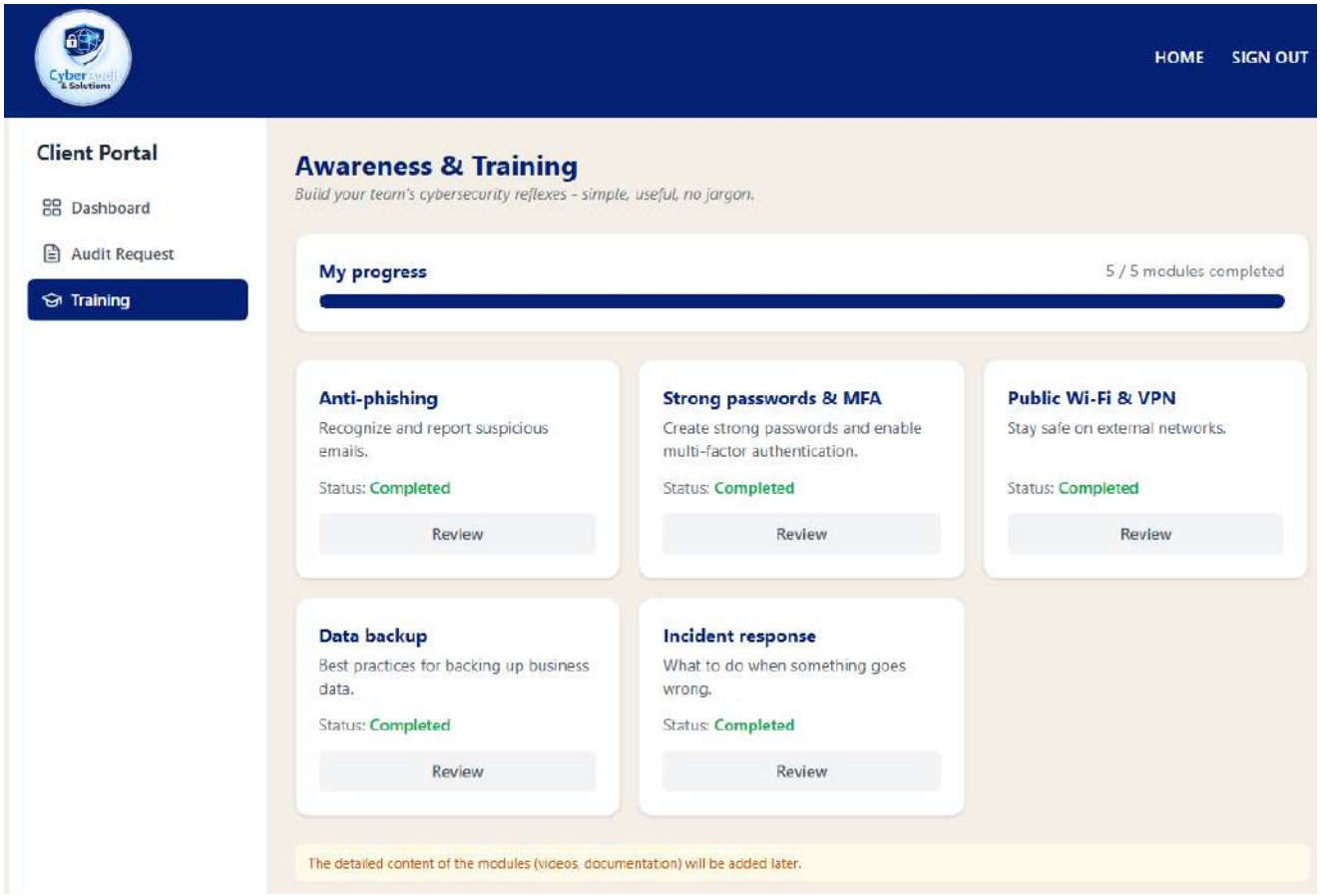
Écran 5 : Formulaire de demande d'audit « Nom d'utilisateur, nom de l'entreprise, choix de la formule et message. Objectif : moins de trois minutes ».

En cliquant sur l'onglet « Audit Request », la cliente accède au formulaire. Elle doit renseigner : son nom d'utilisateur (50 caractères maximum) et le nom de l'entreprise (moins de 50 caractères) ; choisir le pack (Audit / Sécurité / Protection / Premium) ; la sélection d'un pack via le bouton radio affiche ses détails dans le panneau « Included Services » (public visé, périmètre d'analyse, estimation de la durée et tarif) ; et rédiger un message optionnel décrivant la demande souhaitée. Le formulaire se soumet via « Sent the audit request ».



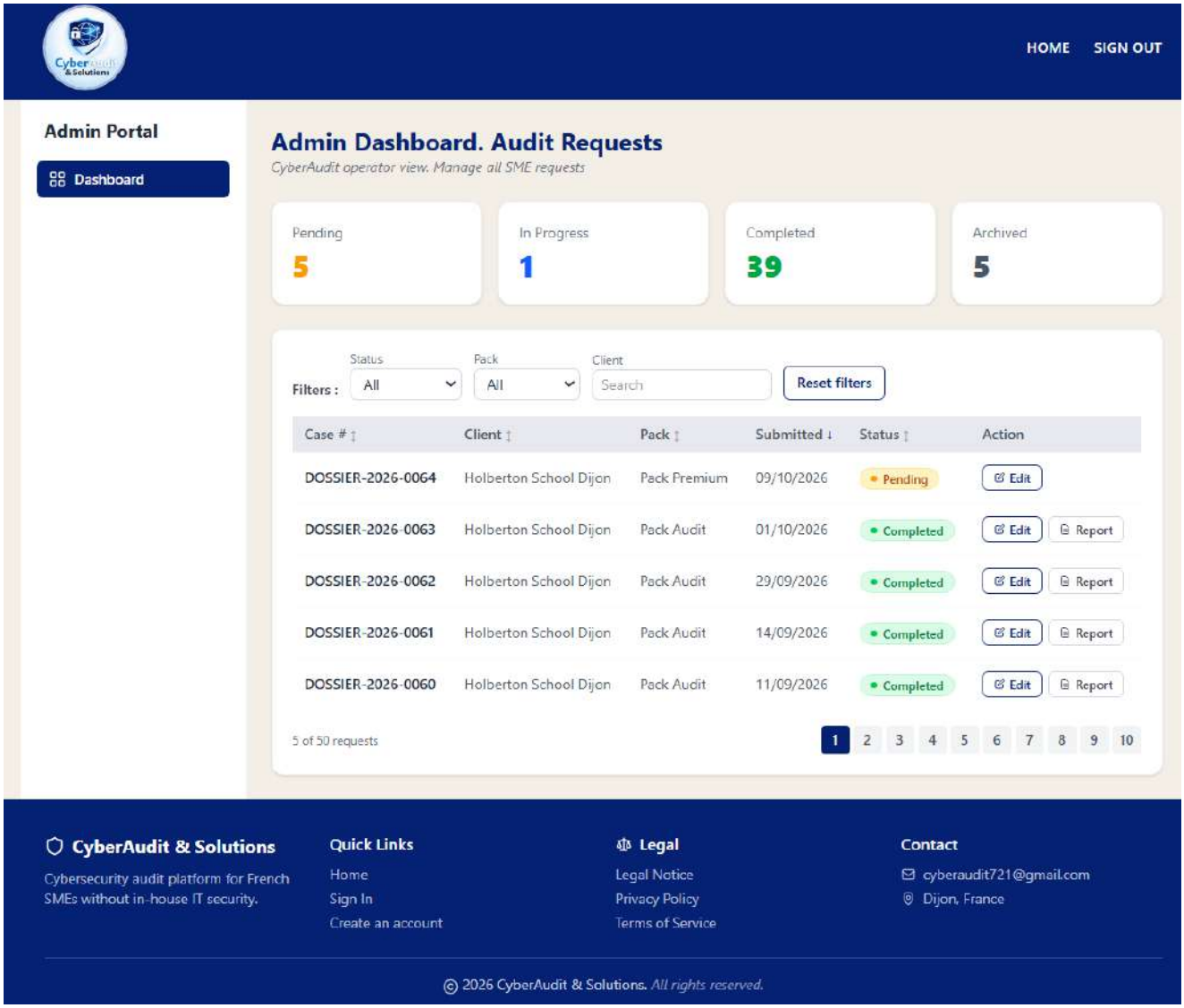
Écran 6 : Confirmation d'envoi d'une demande côté client, récapitulatif de la demande et accusé de réception.

Après l'envoi de la demande, un écran de confirmation reprend le récapitulatif de la demande (numéro de dossier, formule choisie, services inclus, prix, délai estimé et message transmis), et confirme qu'un courriel de confirmation a été envoyé à la cliente par une tâche asynchrone Celery qui appelle l'API du service EmailJS. Le suivi ultérieur du statut (changements successifs, consultation du détail) s'effectue ensuite depuis le tableau de bord et son fil de notifications (écran 4).



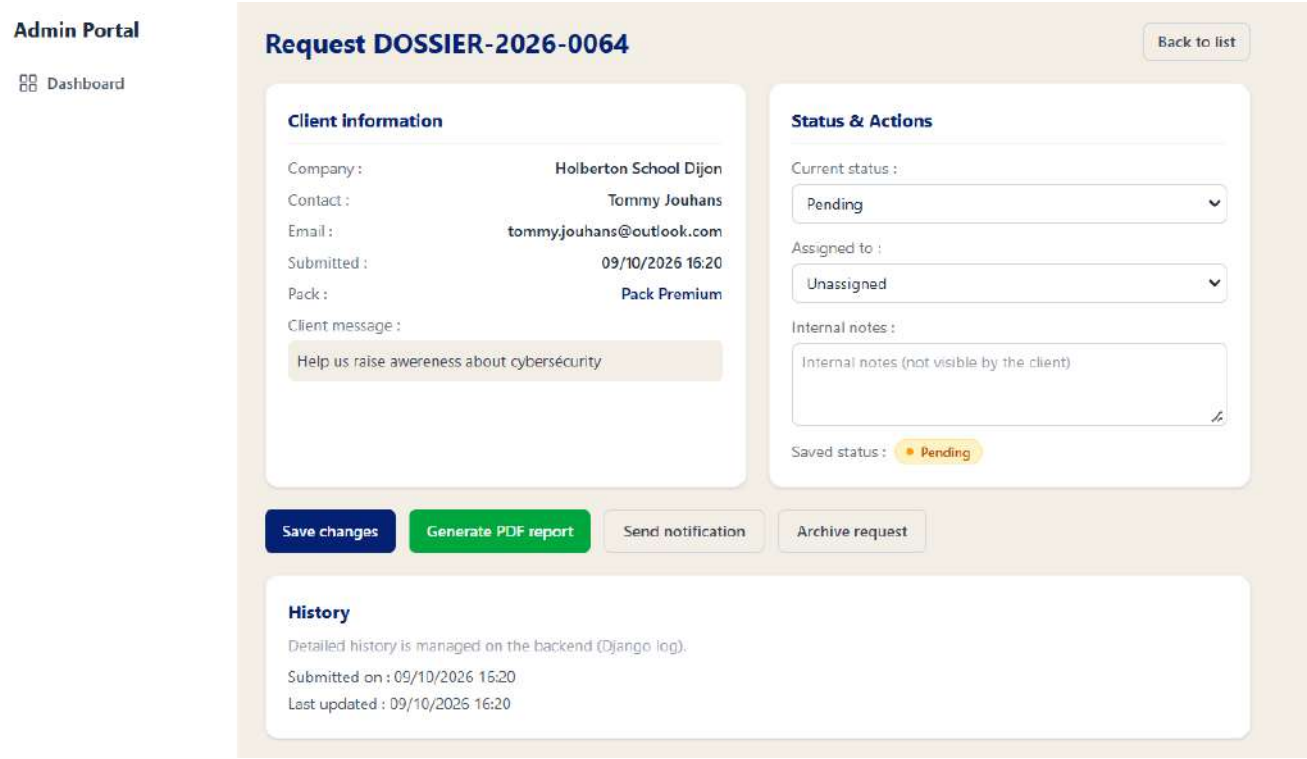
Écran 7 : Modules de sensibilisation à la cybersécurité (hameçonnage, authentification multifacteurs, sauvegarde des données).

Cinq modules de sensibilisation sont proposés : anti-hameçonnage, authentification multifacteurs, sécurité des connexions publiques et Wi-Fi/VPN, sauvegarde des données, réponse aux incidents, avec une barre de progression par utilisateur (ici deux modules sur cinq déjà validés). Ils sont prévus sous différents formats (vidéo, visioconférence, documentation). Cette fonctionnalité, de priorité Should have, a été maquettée pendant la formation ; son développement applicatif complet dépasse le périmètre couvert par le présent titre et sera poursuivi dans une version ultérieure du produit (section 12.1), une gamification des modules avec des cours et vidéo de sensibilisation seront déployé prochainement au-delà du passage du titre RNCP.



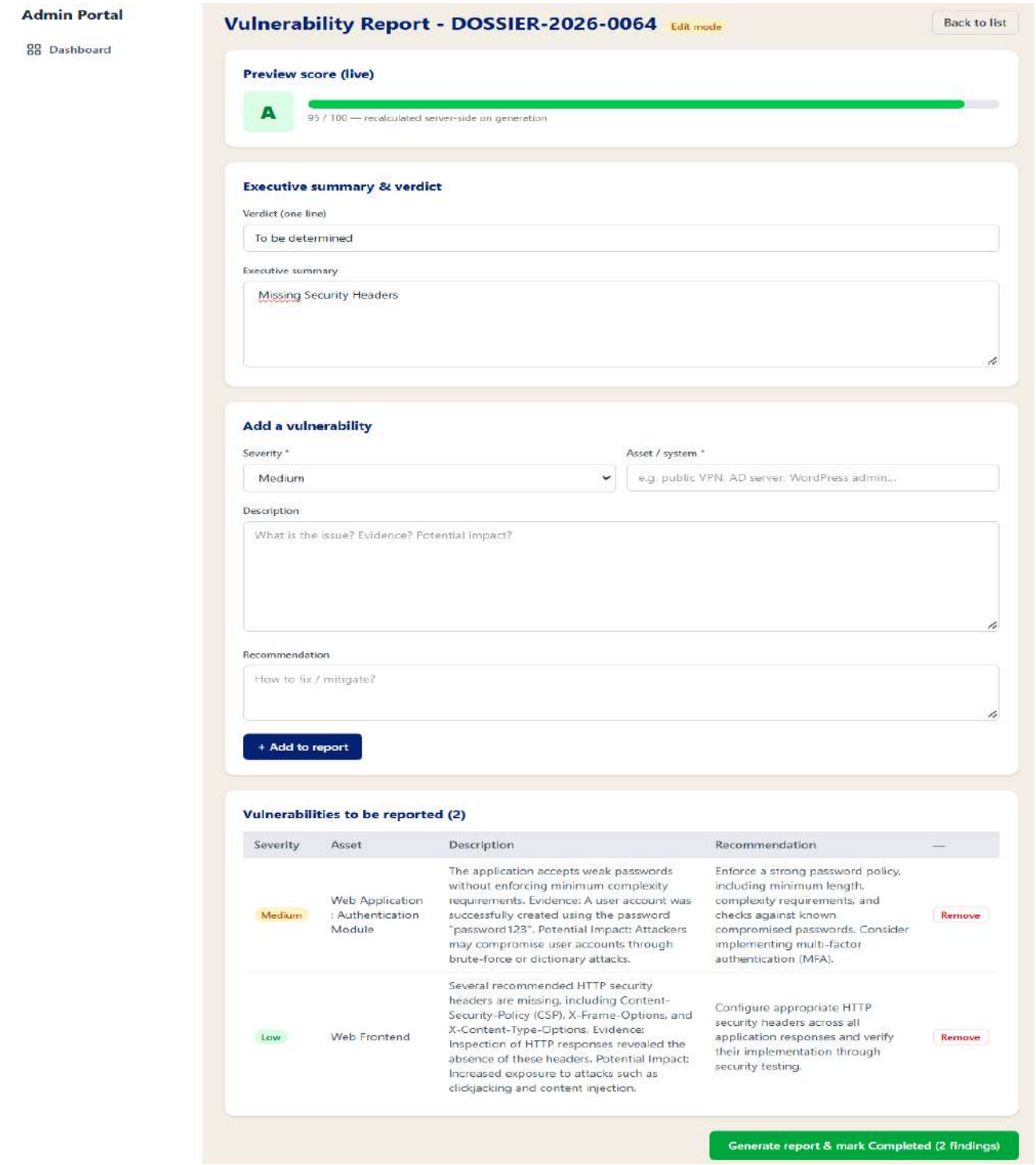
Écran 8 : tableau de bord administrateur en vue globale des demandes avec compteurs par statut et filtres par statut, par formule, par date et par client.

L'expert (administrateur) accède à un tableau de bord rassemblant toutes les demandes reçues, avec des compteurs synthétiques (en attente, en cours, terminées, archivées). Il peut filtrer la liste selon le statut, le pack, la date ou le client, puis consulter et modifier chaque demande en cliquant sur « View / Edit ».



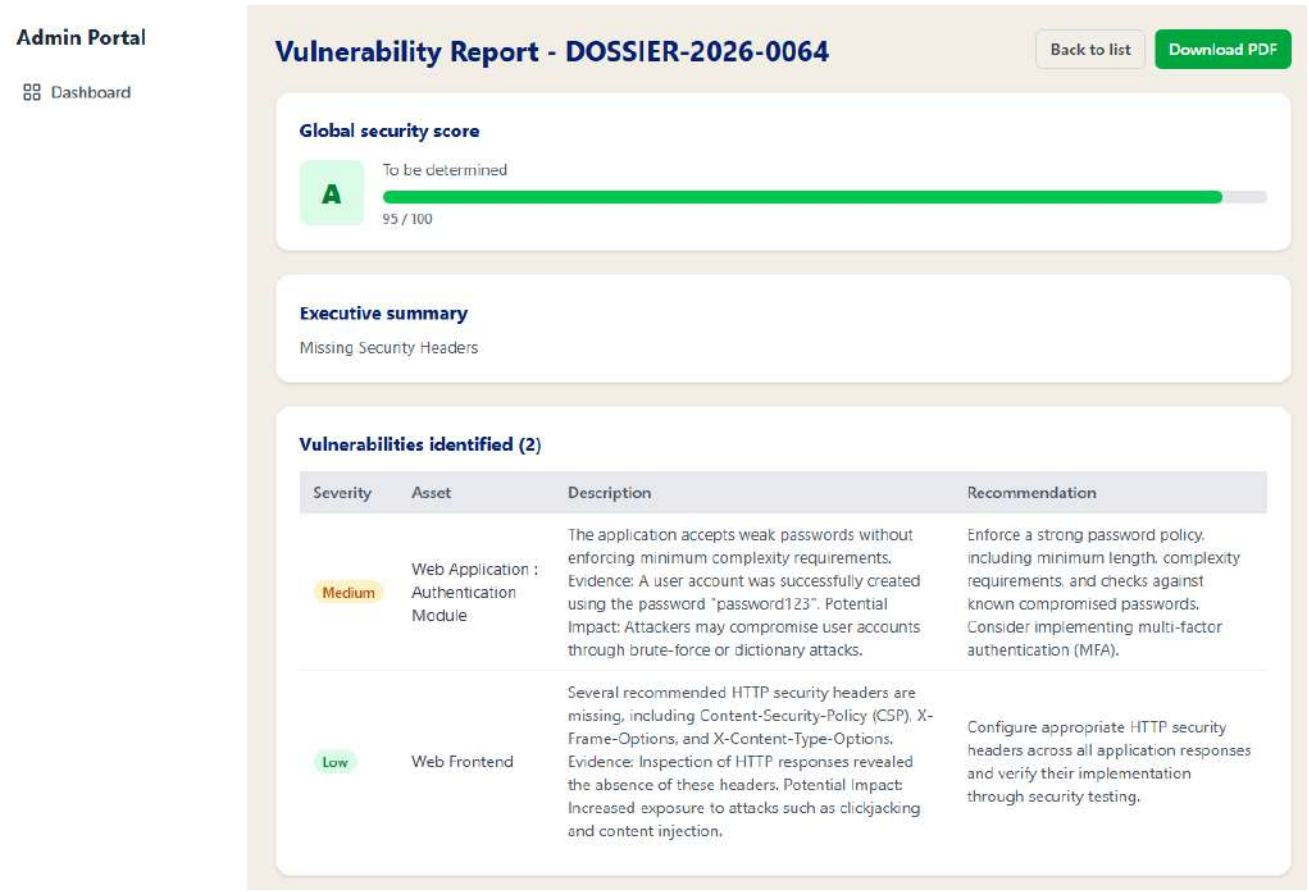
Écran 9 : Détail d'une demande côté administrateur « changement de statut, attribution, notes internes, génération du rapport PDF, envoi de notification, historique immuable ».

Depuis la vue détaillée, l'expert consulte les informations de la cliente, change le statut et l'attribution via le panneau « Status & Actions », ajoute des notes internes, génère un rapport PDF via « Generate PDF report », envoie une notification à la cliente et archive la demande. Un historique immuable (exigence RGPD) trace chaque action horodatée : soumission, envoi de l'accusé Celery, changement de statut, ajout de note.



Écran 10 : Le rapport de vulnérabilité à éditer l'administrateur

L'administrateur peut rédiger un rapport de vulnérabilité en fonction du niveau de la gravité, de l'atout concerné, la description et des recommandations pour le suivi de demande d'audit au client. Une fois que le rapport est rédigé, l'administrateur peut le générer en PDF.



Écran 11 : Aperçu du rapport de vulnérabilités « score de sécurité global, actifs analysés, description, recommandations, plan d'action à 30 jours ».

Le rapport affiche un score de sécurité global sur 100 avec une note synthétique (par exemple A / Acceptable), un résumé exécutif, les vulnérabilités identifiées (niveau de gravité, actif concerné, description et recommandation) et un plan d'action à 30 jours généré automatiquement, téléchargeable au format PDF.

Schéma d'enchaînement des maquettes

L'enchaînement des écrans suit un parcours linéaire pour le client : de la page d'accueil publique (écran 1), l'utilisateur non authentifié est dirigé vers l'inscription (écran 2) ou la connexion (écran 3). Une fois authentifié, il accède à son tableau de bord (écran 4), point d'entrée vers la création d'une nouvelle demande (écran 5), la consultation de la confirmation d'envoi (écran 6), les modules de formation (écran 7) ou le téléchargement d'un rapport disponible (écran 11, depuis le même tableau de bord une fois le rapport généré en écran 10). Côté administrateur, l'enchaînement bascule après authentification vers le tableau de bord global (écran 8), puis vers le détail et le traitement d'une demande individuelle (écran 9), qui déclenche la génération du rapport consulté en écran 10. Ce schéma de navigation a été formalisé et validé collectivement avant le début du développement des interfaces statiques (section 7.4) ; sa version graphique (arborescence des composants et logique de routage) à voir dans le dossier de projet.

Annexe 3 : Schéma entité-association de la base de données

Schéma conceptuel de données de CyberAudit & Solutions, référencé dans l'exemple « Conception et mise en place de la base de données relationnelle » (Activité-type 2).

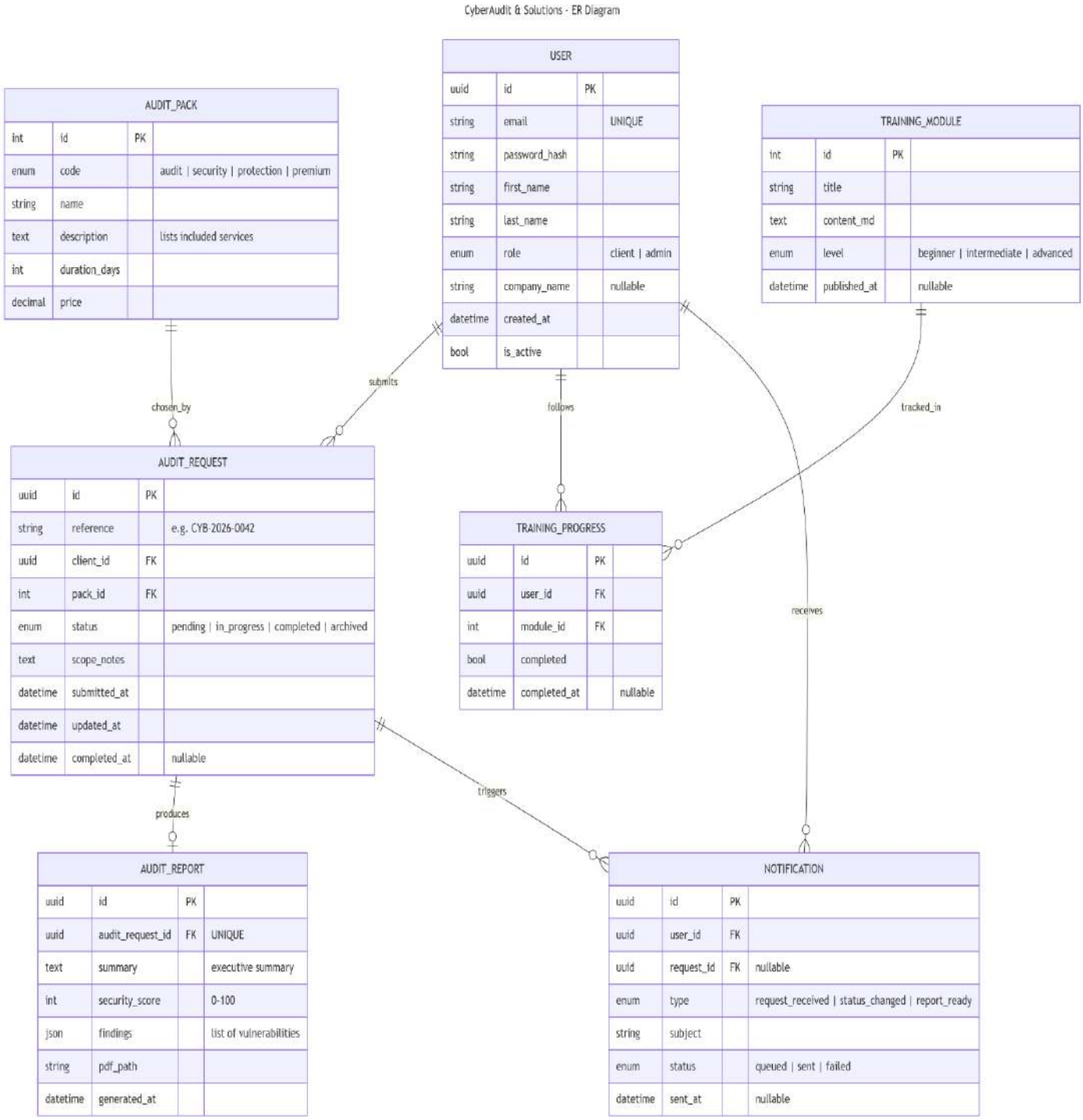


Figure 13 : Schéma entité-association de CyberAudit & Solutions

Annexe 4 : Diagrammes de séquence

Diagramme de séquence illustrant le scénario d'authentification JWT (connexion, validation du jeton, rafraîchissement automatique en cas d'expiration).

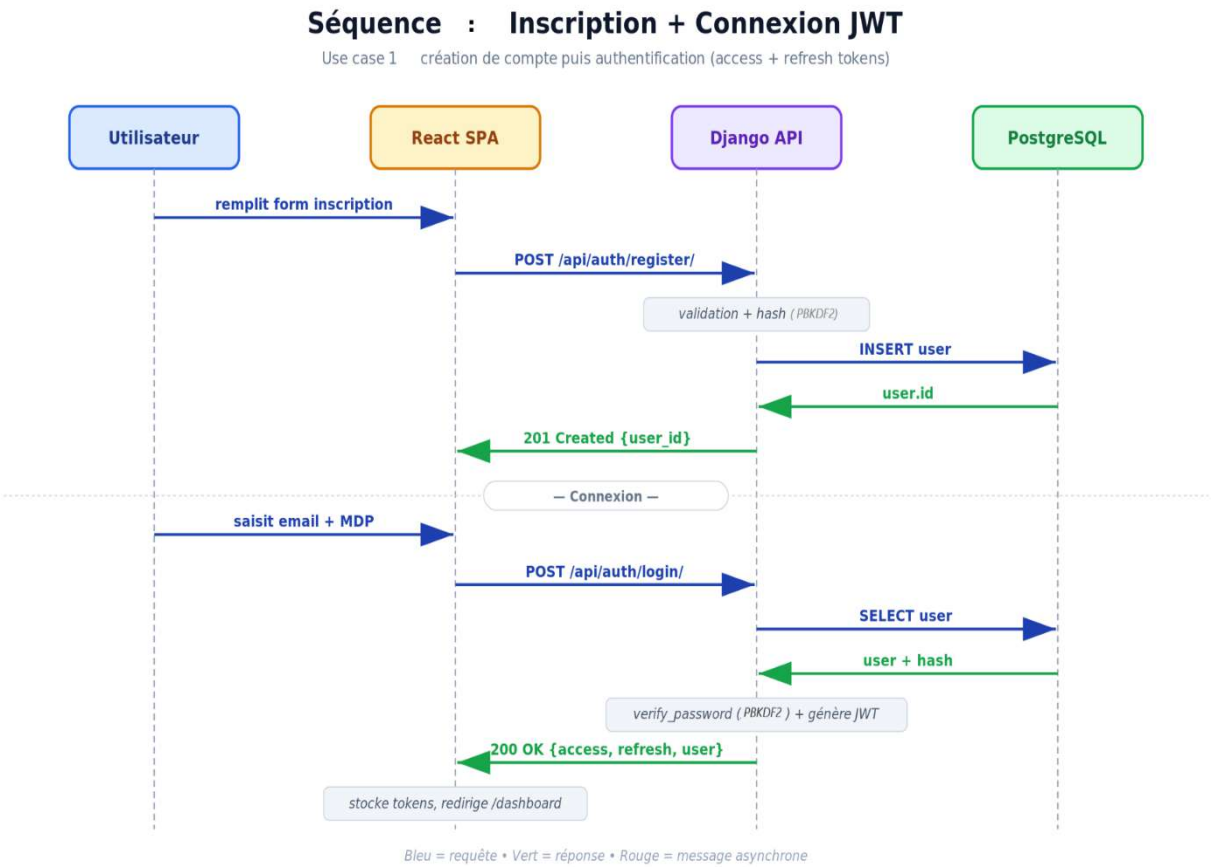


Figure 14 : Séquence d'authentification (login, rafraîchissement de jeton)

L'utilisateur saisit son courriel et son mot de passe ; le front appelle POST /api/auth/register/, puis POST /api/auth/login/. Django valide les identifiants, génère les jetons access et refresh, et les retourne au front, qui les stocke dans **AuthContext** avant de rediriger vers le tableau de bord correspondant au rôle.

Remarque : ce schéma, produit lors de la phase de conception amont, mentionne à titre d'illustration l'algorithme Argon2. L'implémentation finalement retenue pour le hachage des mots de passe est PBKDF2, paramétrage par défaut de Django (cf. section 8.4) ; seule la fonction de hachage diffère, le déroulé du flux d'inscription et de connexion représenté ci-dessus reste inchangé.

Annexe 5 : Extrait de code : contexte d'authentification (AuthContext.jsx)

Contexte React global exposant l'état d'authentification (utilisateur, jeton, rôle) à l'ensemble de l'application, référencé dans l'exemple « Développement de la partie dynamique des interfaces utilisateur » (Activité-type 1).

```jsx
import { createContext, useState, useEffect, useCallback } from "react";
import * as authService from "../services/authService.js";

export const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  // user    : objet utilisateur connecté ou null.
  const [user, setUser]   = useState(null);
  // token   : access token JWT courant ou null.
  const [token, setToken] = useState(null);
  // ready   : true une fois la session initiale vérifiée (évite le flash de redirection).
  const [ready, setReady] = useState(false);

  // ── Restauration de session au montage ──────────────────────
  /* eslint-disable react-hooks/set-state-in-effect */
  useEffect(() => {
    const session = authService.getSession();
    if (session) {
      setUser(session.user);
      setToken(session.access);   // access token (format nouveau)
    }
    setReady(true);
  }, []);
  /* eslint-enable react-hooks/set-state-in-effect */
```

```jsx
  // ── register ────────────────────────────────────────────────
  const register = useCallback(async (form) => {
    // Lance une Error si l'API répond avec une erreur (email déjà pris, etc.)
    const session = await authService.register(form);
    setUser(session.user);
    setToken(session.access);
    return session;
  }, []);


  // ── login ───────────────────────────────────────────────────
  const login = useCallback(async (email, password) => {
    // Lance une Error si les identifiants sont invalides.
    const session = await authService.login(email, password);
    setUser(session.user);
    setToken(session.access);
    return session;
  }, []);


  // ── logout ──────────────────────────────────────────────────
  const logout = useCallback(async () => {
    await authService.logout();
    setUser(null);
    setToken(null);
  }, []);
```

```
  // ── hasRole ──────────────────────────────────────────────────────────
  const hasRole = useCallback(
    (role) => Boolean(user) && user.role === role,
    [user],
  );

  const value = {
    user,
    token,
    ready,
    isAuthenticated: Boolean(user),
    login,
    register,
    logout,
    hasRole,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}
```

Annexe 6 : Extrait de code : intercepteur Axios (rafraîchissement automatique du jeton JWT)

Intercepteur de réponse Axios déclenchant un rafraîchissement automatique du jeton JWT expiré, référencé dans l'exemple « Développement de la partie dynamique des interfaces utilisateur » (Activité-type 1).

```
// ── Intercepteur requête : ajoute Authorization: Bearer <access> ──────────
api.interceptors.request.use((config) => {
  // Ne pas injecter le token sur les routes d'authentification elles-mêmes
  const isAuthRoute =
    config.url?.includes("token/refresh") ||
    config.url?.includes("login") ||
    config.url?.includes("register");

  if (!isAuthRoute) {
    const raw = sessionStorage.getItem(SESSION_KEY);
    if (raw) {
      const session = JSON.parse(raw);
      if (session?.access) {
        config.headers.Authorization = `Bearer ${session.access}`;
      }
    }
  }
  return config;
});
```

```
// ── Intercepteur réponse : refresh automatique si 401 ──────────────────────
api.interceptors.response.use(
  (response) => {
    // Auto-unwrap DRF pagination : {count, next, previous, results: [...]} -> [...]
    if (
      response.data
      && typeof response.data === "object"
      && Array.isArray(response.data.results)
      && "count" in response.data
    ) {
      response.data = response.data.results;
    }
    return response;
  },
  async (error) => {
    const original = error.config;

    const is401 = error.response?.status === 401;
    const alreadyRetried = original._retry;
    const isAuthRoute =
      original.url?.includes("token/refresh") ||
      original.url?.includes("login") ||
      original.url?.includes("register") ||
      original.url?.includes("logout");   // logout ne doit jamais être retenté
```

```
if (is401 && !alreadyRetried && !isAuthRoute) {
  original._retry = true;

  try {
    const raw = sessionStorage.getItem(SESSION_KEY);
    if (!raw) throw new Error("Pas de session");

    const session = JSON.parse(raw);
    if (!session?.refresh) throw new Error("Pas de refresh token");

    // Appel direct axios (pas l'instance api) pour éviter une boucle
    const { data } = await axios.post(`${BASE_URL}/auth/token/refresh/`, {
      refresh: session.refresh,
    });

    // Mise à jour du token en sessionStorage
    const updated = {
      ...session,
      access: data.access,
      refresh: data.refresh ?? session.refresh,
    };
    sessionStorage.setItem(SESSION_KEY, JSON.stringify(updated));

    // Rejoue la requête originale avec le nouveau token
    original.headers.Authorization = `Bearer ${data.access}`;
    return api(original);
  } catch (_refreshError) {
    // Refresh échoué → déconnexion forcée
    sessionStorage.removeItem(SESSION_KEY);
    window.location.href = "/login";
    return Promise.reject(_refreshError);
  }
}
```

```
    return Promise.reject(error);
  },
);

export default api;
```

Annexe 7 : Extrait du script de création de la base de données (migration Django)

Extrait de migration Django créant la table **notifications**, utilisée pour journaliser les envois d'e-mails transactionnels (confirmation de demande, changement de statut) déclenchés par Celery.

```python
class Migration(migrations.Migration):

    initial = True

    dependencies = [
        ('audits', '0002_seed_packs'),
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]

    operations = [
        migrations.CreateModel(
            name='Notification',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True, serialize=False)),
                ('type', models.CharField(choices=[('request_received', 'Request received'), ('status_changed', 'Status changed'), ('report_ready', 'Re
                ('subject', models.CharField(max_length=255)),
                ('message', models.TextField()),
                ('status', models.CharField(choices=[('queued', 'Queued'), ('sent', 'Sent'), ('failed', 'Failed')], db_index=True, default='queued', ma
                ('sent_at', models.DateTimeField(blank=True, null=True)),
                ('error_message', models.TextField(blank=True)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('request', models.ForeignKey(blank=True, null=True, on_delete=django.db.models.deletion.CASCADE, related_name='notifications', to='aud
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='notifications', to=settings.AUTH_USER_MODEL))
            ],
            options={
                'db_table': 'notifications',
                'ordering': ['-created_at'],
                'indexes': [models.Index(fields=['user', '-created_at'], name='notificatio_user_id_611c58_idx')],
            },
        ),
    ]
```

Annexe 8 : Top 10 des OSWAP



Le **Top 10 OWASP** présente les dix principales catégories de risques de sécurité auxquels les applications web sont exposées :

1. **Contrôle d'accès défaillant** : accès non autorisé à des données ou fonctionnalités.
2. **Défaillances cryptographiques** : protection insuffisante des données sensibles.
3. **Injection** : insertion de commandes malveillantes dans une application.
4. **Conception non sécurisée** : failles liées à une mauvaise conception de la sécurité.
5. **Mauvaises configurations de sécurité** : paramètres incorrects ou protections insuffisantes.
6. **Composants vulnérables ou obsolètes** : utilisation de bibliothèques et logiciels présentant des failles.
7. **Défaillances d'identification et d'authentification** : mécanismes de connexion insuffisants.
8. **Défaillances d'intégrité des logiciels et des données** : absence de garanties suffisantes sur l'intégrité des éléments utilisés.
9. **Journalisation et surveillance insuffisantes** : difficulté à détecter les attaques et les comportements suspects.
10. **Falsification de requêtes côté serveur** : exploitation du serveur pour accéder à des ressources internes non autorisées.

Lien avec le projet CyberAudit & Solutions

Le Top 10 OWASP constitue une référence importante pour identifier et prévenir les vulnérabilités des applications web. Dans le cadre de **CyberAudit & Solutions**, il permet de sensibiliser les utilisateurs aux

risques informatiques et de guider les audits de sécurité afin de renforcer la protection des données et la fiabilité de la plateforme.

Objectif : anticiper les cyberattaques, réduire les vulnérabilités et développer des applications web plus sûres.